SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Property Platform (Multi-Tenant SaaS + AI-Driven Marketplace)

---

1. INTRODUCTION

1.1 Purpose

This document defines the functional and non-functional requirements for a **scalable property marketplace platform** that connects property agents and buyers through a high-performance, search-first experience.

1.2 Scope

The system includes:

- Web platform (SEO + responsive UI)
- Mobile application (built with .NET MAUI)
- Backend services (multi-tenant architecture)
- AI-based recommendation engine
- Advanced search (PostgreSQL + Elasticsearch)

The platform supports:

- Agent onboarding and listing management
- User property search and discovery
- Lead generation and tracking
- Monetization via subscriptions and featured listings

1.3 Definitions

Term	Meaning
Tenant	Agent account
Listing	Property record
Lead	User contacting agent
CTR	Click-through rate
Conversion	Lead generated

1.4 References

- ISO/IEC/IEEE 29148
- .NET
- PostgreSQL
- Elasticsearch

---

2. OVERALL DESCRIPTION

2.1 Product Perspective

The system is a **multi-tenant SaaS marketplace** consisting of:

- Web UI (Razor + Tailwind)
- Mobile App (MAUI)
- Backend API (.NET)
- Search Engine (Elastic)
- Cache Layer (Redis)
- CDN (media delivery)

2.2 Product Functions

- Property listing CRUD
- Search and filtering
- AI recommendations
- Lead generation and tracking
- Agent onboarding
- Referral system
- Monetization system

2.3 User Classes

User	Description
Visitor	Browses listings
Buyer	Searches and contacts agent
Agent (Tenant)	Creates listings
Admin	Manages system

2.4 Operating Environment

- Browser (desktop + mobile)
- Mobile app (Android/iOS via MAUI)
- Cloud deployment (Docker + Kubernetes)

2.5 Constraints

- Use open-source stack
- Must support multi-tenancy
- Must scale to millions of users
- Must support SEO

2.6 Assumptions

- Users access via mobile-first experience
- High traffic expected in urban areas (e.g., KL)

---

3. SYSTEM ARCHITECTURE

3.1 High-Level Architecture

```
[ CDN ]
  ↓
[ Load Balancer ]
  ↓
[ API Instances ]
  ↓
[ Redis Cache ]
  ↓
[ PostgreSQL Cluster ]
  ↓
[ Elasticsearch Cluster ]
```

3.2 Architectural Style

- Modular Monolith → Microservices-ready
- Event-driven (future)

3.3 Key Components

- API Layer (.NET)
- Search Service (Elastic)
- AI Recommendation Engine
- Media Storage (S3/CDN)
- Queue System (future)

---

4. FUNCTIONAL REQUIREMENTS

4.1 Authentication & Security

- FR-1: User login via JWT
- FR-2: Refresh token support
- FR-3: Password hashing required
- FR-4: Tenant isolation enforced

4.2 Agent Onboarding

- FR-5: Agent signup with minimal fields
- FR-6: Guided listing creation flow
- FR-7: Listing must be publishable within 3 minutes

4.3 Property Listing

- FR-8: Agent can create, edit, delete listings
- FR-9: Listing includes:
    - Title, price, location
    - Images, floor plan
    - Description

4.4 Search & Filtering

- FR-10: Keyword search
- FR-11: Filter by:
    - Location
    - Price range
- FR-12: Pagination support
- FR-13: Full-text search (PostgreSQL)

4.5 Advanced Search (Elastic)

- FR-14: Fuzzy search
- FR-15: Ranking relevance
- FR-16: Scalable search for 10M+ listings

4.6 AI Recommendation

- FR-17: Show recommended listings
- FR-18: Track user behavior
- FR-19: Score listings based on:
    - Similarity
    - Popularity
    - Freshness

4.7 Lead Generation

- FR-20: User can contact agent
- FR-21: System logs leads
- FR-22: Lead scoring implemented

4.8 Referral System

- FR-23: Agent referral tracking
- FR-24: Reward system for referrals

4.9 Monetization

- FR-25: Subscription plans
- FR-26: Featured listing boost
- FR-27: Lead-based pricing (optional)

4.10 Mobile App (MAUI)

- FR-28: Search properties
- FR-29: View listings
- FR-30: Offline cache support

---

5. NON-FUNCTIONAL REQUIREMENTS

5.1 Performance

Metric	Target
API response	< 200 ms
Search	< 300 ms
Page load	< 1.5 s

5.2 Scalability

- Must support:
  - 1M+ listings
  - 100k+ daily users
- Horizontal scaling required

5.3 Availability

- 99.9% uptime
- Failover support

5.4 Security

- HTTPS enforced
- JWT authentication
- Input validation

5.5 Maintainability

- Modular architecture
- Clean code principles
- API documentation (Swagger)

5.6 Usability

- Mobile-first design
- < 3 steps to search
- < 3 minutes onboarding

---

6. DATA REQUIREMENTS

6.1 Core Entities

- Tenant (Agent)
- PropertyListing
- Media
- Lead
- UserEvents
- Referral

6.2 Partitioning Strategy

- Partition by location or tenant

6.3 Indexing

- Price
- Location
- CreatedAt
- Search vector

---

7. AI & ANALYTICS

7.1 User Event Tracking

- View
- Click
- Contact

7.2 Recommendation Engine

- Phase 1: Rule-based
- Phase 2: Behavior-based
- Phase 3: Collaborative filtering

7.3 Ranking Algorithm

Ranking =
  Freshness + Conversion + Agent Quality + Boost

---

8. DEVOPS & DEPLOYMENT

8.1 Infrastructure

- Docker containers
- Kubernetes orchestration

8.2 CI/CD

- Automated build
- Automated deployment

8.3 Monitoring

- Logging (Serilog)
- Metrics tracking

---

9. BUSINESS REQUIREMENTS

9.1 Monetization

- Subscription tiers
- Featured listings
- Lead monetization

9.2 Growth Strategy

- Agent onboarding
- Referral system
- SEO traffic
- Social media marketing

---

10. RISKS

10.1 Technical Risks

- Search scalability
- Data leakage (multi-tenant)

10.2 Business Risks

- Lack of listings
- Low user adoption

---

11. FUTURE ROADMAP

- AI recommendation improvement
- Map-based search
- Microservices migration
- Regional expansion

TECHNICAL ARCHITECTURE DOCUMENT (TAD)

Property Platform (Scalable Multi-Tenant SaaS)

Aligned with
ISO/IEC/IEEE 29148

---

1. ARCHITECTURE OVERVIEW

1.1 System Context

```
Users (Web + Mobile)
      ↓
  CDN (Images)
      ↓
 Load Balancer (Nginx)
      ↓
 API Layer (.NET)
      ↓
┌──────────────────┬──────────────────┐
│            │            │
Redis Cache   PostgreSQL     Elasticsearch
```

1.2 Architectural Style

- **Primary:** Modular Monolith
- **Target:** Microservices-ready
- **Pattern:** Clean Architecture + DDD-lite

---

2. COMPONENT ARCHITECTURE

2.1 Web Layer (Presentation)

Tech:

- Razor Pages
- Tailwind CSS
- Optional: HTMX / Alpine.js

Responsibilities:

- SEO rendering
- UI interaction
- API consumption

2.2 Mobile Layer

Tech:

- .NET MAUI

Pattern:

- MVVM
- DI (built-in container)

2.3 API Layer

Tech:

- .NET

Responsibilities:

- Authentication
- Business logic
- Multi-tenant enforcement

- Data orchestration

2.4 Application Layer

- Use cases (services)
- DTO mapping
- Validation

2.5 Domain Layer

- Entities
- Aggregates
- Business rules

2.6 Infrastructure Layer

Includes:

- Database (PostgreSQL)
- Cache (Redis)
- Search (Elasticsearch)
- File storage (S3/CDN)

---

3. DATA ARCHITECTURE

3.1 Database Design

DB:

- PostgreSQL

Key Design:

- Multi-tenant via `TenantId`
- Partitioning by location

3.2 Read/Write Separation

Write → Primary DB
Read → Read Replica

3.3 Search Architecture

Engine:

- Elasticsearch

Strategy:

- DB = source of truth
- Elastic = search index

3.4 Data Flow (Sync)

API → DB → Event → Elastic Index

---

4. MULTI-TENANCY DESIGN

4.1 Strategy

- Shared DB
- Tenant isolation via `TenantId`

4.2 Enforcement

- Middleware extracts TenantId from JWT
- EF Core global query filter

4.3 Risk Mitigation

- No query without TenantId
- Automated tests for isolation

---

5. AUTHENTICATION ARCHITECTURE

5.1 Mechanism

- JWT (Access Token)
- Refresh Token (DB stored)

5.2 Flow

Login → JWT issued → stored in client
→ API validates → Tenant resolved

---

6. SEARCH ARCHITECTURE

6.1 Hybrid Strategy

Stage	Tech
MVP	PostgreSQL FTS
Scale	Elasticsearch

6.2 Query Flow

User Query → API → Elastic
→ fallback DB (if needed)

6.3 Ranking Inputs

- Relevance
- Price match
- Location match
- Popularity

---

7. AI RECOMMENDATION ARCHITECTURE

7.1 Components

- Event Tracker
- Recommendation Engine
- Cache Layer

7.2 Flow

User Actions → Event DB → Scoring Engine
→ Cache → API → UI

7.3 Evolution

- Phase 1: Rule-based
- Phase 2: Behavior scoring
- Phase 3: Collaborative filtering

---

8. FILE STORAGE ARCHITECTURE

8.1 Strategy

- Upload → Storage (S3/local)
- Serve via CDN

8.2 Flow

Upload → API → Storage → CDN → User

---

9. CACHING STRATEGY

9.1 Layers

Layer	Purpose
CDN	Images
Redis	Search results
App memory	Hot data

9.2 Cache Pattern

- Cache-aside
- TTL-based invalidation

---

10. PERFORMANCE STRATEGY

10.1 Targets

Metric	Target
API	<200ms
Search	<300ms
Page	<1.5s

10.2 Techniques

- Pagination
- Indexing
- Async processing
- Compression (WebP images)

---

11. DEVOPS ARCHITECTURE

11.1 Containerization

- Dockerized services

11.2 Orchestration

- Kubernetes

11.3 CI/CD

- GitHub Actions pipeline

11.4 Deployment Flow

Code → Build → Test → Docker → Deploy → Monitor

---

12. OBSERVABILITY

12.1 Logging

- Serilog (structured logs)

12.2 Monitoring

- Request latency
- Error rate
- DB performance

12.3 Alerts

- High error rate

- Slow queries

---

13. SECURITY ARCHITECTURE

13.1 Controls

- HTTPS
- JWT validation
- Input sanitization
- Rate limiting

13.2 File Upload Security

- Validate MIME type
- Size limit
- Virus scan (future)

---

14. SCALABILITY ROADMAP

Stage 1

- Monolith + Redis

Stage 2

- Read replicas + CDN

Stage 3

- Microservices + Elastic cluster

---

15. RISKS & MITIGATIONS

Technical

Risk	Mitigation
Slow search	Elastic + caching
Data leak	Tenant filter enforcement

Operational

Risk	Mitigation
High load	Auto-scaling
Storage cost	CDN + compression

FINAL ARCHITECT ASSESSMENT

This architecture is:

- **Production-ready**
- Scalable to **millions of users**
- Flexible for future microservices

Good—this is the right next step. I'll translate your architecture into a **production-grade ERD (Entity Relationship Design)** with:

- Finalized schema (optimized for scale)
- Multi-tenant enforcement
- AI + analytics + growth features included
- Ready for **PostgreSQL + Elasticsearch sync**

FINAL PRODUCTION ERD (TEXTUAL + LOGICAL DESIGN)

1. HIGH-LEVEL ENTITY MAP

```
Tenant (Agent)
  │
  ├── AgentProfile
  ├── Subscription
  ├── PropertyListing
  │     ├── PropertyMedia
  │     ├── PropertyFeature
  │     ├── FloorPlan
  │     ├── Lead
  │     └── ListingAnalytics
  │
  ├── Referral
  └── RefreshToken


User
```

└── UserEvents

---

2. CORE TABLES (FINAL DESIGN)

2.1 TENANTS (AGENTS)

```
CREATE TABLE Tenants (
    TenantId UUID PRIMARY KEY,
    Email VARCHAR(255) UNIQUE NOT NULL,
    PasswordHash TEXT NOT NULL,
    CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

2.2 AGENT PROFILE

```
CREATE TABLE AgentProfiles (
    AgentId UUID PRIMARY KEY,
    TenantId UUID NOT NULL,
    Name VARCHAR(255),
    REN_ID VARCHAR(100),
    Phone VARCHAR(50),
    OfficeAddress TEXT,
```

```
    ProfilePhotoUrl TEXT,
    CompanyLogoUrl TEXT,
    FOREIGN KEY (TenantId) REFERENCES Tenants(TenantId)
);
```

2.3 PROPERTY LISTING (PARTITIONED READY)

```
CREATE TABLE PropertyListings (
    ListingId UUID PRIMARY KEY,
    TenantId UUID NOT NULL,
    Title VARCHAR(255),
    Description TEXT,
    Location VARCHAR(255),
    Price NUMERIC(18,2),
    Status VARCHAR(50),
    CreatedAt TIMESTAMP,
    UpdatedAt TIMESTAMP
);
```

2.4 PROPERTY MEDIA

```
CREATE TABLE PropertyMedia (
    MediaId UUID PRIMARY KEY,
```

```
    ListingId UUID NOT NULL,
    MediaUrl TEXT,
    MediaType VARCHAR(50),
    SortOrder INT
);
```

2.5 PROPERTY FEATURES

```
CREATE TABLE PropertyFeatures (
    FeatureId UUID PRIMARY KEY,
    ListingId UUID NOT NULL,
    Key VARCHAR(100),
    Value VARCHAR(255)
);
```

2.6 FLOOR PLAN

```
CREATE TABLE FloorPlans (
    FloorPlanId UUID PRIMARY KEY,
    ListingId UUID NOT NULL,
    ImageUrl TEXT,
    Description TEXT
);
```

3. LEAD & CONVERSION SYSTEM

3.1 LEADS

```
CREATE TABLE Leads (
    LeadId UUID PRIMARY KEY,
    ListingId UUID NOT NULL,
    TenantId UUID NOT NULL,
    ContactType VARCHAR(50), -- WhatsApp, Form
    CreatedAt TIMESTAMP
);
```

3.2 LISTING ANALYTICS

```
CREATE TABLE ListingAnalytics (
    AnalyticsId UUID PRIMARY KEY,
    ListingId UUID NOT NULL,
    Views INT DEFAULT 0,
    Clicks INT DEFAULT 0,
    Contacts INT DEFAULT 0,
    UpdatedAt TIMESTAMP
```

```
);
```

---

4. AI & USER TRACKING

4.1 USERS (BUYERS)

```
CREATE TABLE Users (
    UserId UUID PRIMARY KEY,
    Email VARCHAR(255),
    CreatedAt TIMESTAMP
);
```

4.2 USER EVENTS (CRITICAL FOR AI)

```
CREATE TABLE UserEvents (
    EventId UUID PRIMARY KEY,
    UserId UUID,
    ListingId UUID,
    EventType VARCHAR(50), -- VIEW, CLICK, CONTACT
    CreatedAt TIMESTAMP
);
```

5. REFERRAL SYSTEM

5.1 REFERRALS

```
CREATE TABLE Referrals (
    ReferralId UUID PRIMARY KEY,
    ReferrerTenantId UUID,
    NewTenantId UUID,
    CreatedAt TIMESTAMP
);
```

6. SUBSCRIPTION & MONETIZATION

6.1 SUBSCRIPTIONS

```
CREATE TABLE Subscriptions (
    SubscriptionId UUID PRIMARY KEY,
    TenantId UUID,
    Plan VARCHAR(50),
    StartDate TIMESTAMP,
    EndDate TIMESTAMP
```

```
);
```

6.2 FEATURED LISTINGS

```
CREATE TABLE FeaturedListings (
    FeatureId UUID PRIMARY KEY,
    ListingId UUID,
    BoostLevel INT,
    StartDate TIMESTAMP,
    EndDate TIMESTAMP
);
```

---

7. AUTHENTICATION

7.1 REFRESH TOKENS

```
CREATE TABLE RefreshTokens (
    TokenId UUID PRIMARY KEY,
    TenantId UUID,
    Token TEXT,
    ExpiryDate TIMESTAMP,
    IsRevoked BOOLEAN
```

);

---

8. RELATIONSHIPS SUMMARY

Relationship	Type
Tenant → Listings	1:N
Listing → Media	1:N
Listing → Leads	1:N
Listing → Analytics	1:1
User → Events	1:N
Tenant → Subscription	1:1

---

9. INDEXING STRATEGY (CRITICAL)

```
CREATE INDEX idx_listing_tenant ON PropertyListings(TenantId);

CREATE INDEX idx_listing_price ON PropertyListings(Price);

CREATE INDEX idx_listing_location ON PropertyListings(Location);
```

```
CREATE INDEX idx_events_listing ON UserEvents(ListingId);
```

---

10. SCALING DESIGN

10.1 PARTITIONING

PARTITION BY LIST (Location)

10.2 HOT DATA

- Cache:
  - popular listings
  - recommendations

10.3 ELASTICSEARCH SYNC

DB → Event → Elastic Index

Below is a **complete Business Plan & Go-To-Market (GTM) Documentation** that consolidates everything we've designed—product, tech, growth, monetization, and execution. This is written so you can use it for:

- Internal alignment
- Execution roadmap
- Investor sharing

BUSINESS PLAN & GO-TO-MARKET DOCUMENT

Property Platform (AI-Driven Real Estate Marketplace)

---

1. EXECUTIVE SUMMARY

1.1 Vision

Build a **high-performance property marketplace** that connects buyers directly with agents, delivering **better leads, faster search, and higher conversion** than traditional platforms.

1.2 Mission

- Provide agents with **direct, high-quality leads**
- Provide users with **fast, relevant property discovery**
- Build a **scalable SaaS platform** for long-term growth

1.3 Problem Statement

Current platforms (e.g. PropertyGuru, iProperty):

- Charge high fees
- Do not guarantee lead quality
- Have cluttered UX
- Slow search experience

1.4 Solution

A platform that offers:

- Direct agent-to-buyer connection
- Fast, search-first UI
- AI-driven recommendations
- Lower cost and better ROI for agents

---

2. PRODUCT OVERVIEW

2.1 Core Features

For Users (Buyers)

- Search properties (fast, filtered)
- View listings (images, price, details)
- Contact agent directly (WhatsApp / form)

For Agents (Tenants)

- Create and manage listings
- Upload images and floor plans
- Receive leads directly
- Track listing performance

System Capabilities

- Multi-tenant SaaS
- AI recommendation engine
- Advanced search (Elastic-ready)
- Mobile app (MAUI)

2.2 Unique Value Proposition

Feature	Benefit
Direct leads	No middleman cost
Fast UI	Better user retention
AI recommendation	Higher conversion
Simple onboarding	Faster agent adoption

---

3. MARKET STRATEGY

3.1 Target Market

Phase 1:

- Kuala Lumpur (high density, high demand)

Phase 2:

- Malaysia (nationwide)

Phase 3:

- Southeast Asia expansion

3.2 Target Users

Segment	Description
Agents	Property agents seeking leads
Buyers	Individuals searching for property
Investors	Property investors

---

4. GO-TO-MARKET STRATEGY

4.1 Phase 1: Supply First (0–500 Listings)

Strategy:

- Manual agent onboarding
- Direct outreach via:
    - Facebook groups
    - WhatsApp groups
    - Listings on PropertyGuru and iProperty

Execution:

- Daily outreach: 50 agents
- Conversion target: 20%
- Weekly onboarding: 50 agents

4.2 Phase 2: Demand Generation

Channels:

1. SEO

- Pages like:
    - “Condo for sale KL”
    - “Apartment Cheras”

2. Social Media

- TikTok / Reels:
    - Property tours
    - Investment insights

3. Paid Ads

- Google Ads (RM50–RM100/day)

4.3 Phase 3: Growth Loop

Listings → SEO → Users → Leads → Agents stay → More listings

---

5. AGENT ONBOARDING STRATEGY

5.1 Objective

Reduce friction to:

< 3 minutes to publish first listing

5.2 Flow

1. Simple signup (email + password)
2. Instant dashboard
3. Guided listing creation
4. Publish
5. Share link

5.3 Success Metric

- % of agents creating 1 listing within 24 hours
    - Target: 40–60%

---

6. MONETIZATION MODEL

6.1 Revenue Streams

1. Subscription Plans

Plan	Price	Features
Basic	RM49	Limited listings

Pro	RM99	More listings
Premium	RM199	Unlimited + boost

---

2. Featured Listings (Primary Revenue Driver)

- Pay to boost visibility
- Appears on:
    - Homepage
    - Top search

---

3. Lead-Based Pricing (Optional)

- Charge per qualified lead

---

4. Ads / Developer Promotion

- Property developers promote projects

---

7. GROWTH ENGINE

7.1 Referral System

- Agents refer agents
- Rewards:
    - Free boost
    - Premium access

7.2 Viral Mechanism

- Share listing → WhatsApp
- Buyers visit → become users

7.3 KPI Targets

Metric	Target
Listings	500 (Month 1)
Agents	100
Daily users	1,000
Conversion rate	>5%

---

8. AI STRATEGY

8.1 Recommendation Engine

Phase 1:

- Rule-based

Phase 2:

- Behavior-based scoring

Phase 3:

- Collaborative filtering

8.2 Goal

- Increase engagement
- Improve lead conversion

---

9. TECHNOLOGY STRATEGY

9.1 Stack

- Backend: .NET
- Database: PostgreSQL
- Search: Elasticsearch
- Cache: Redis
- Mobile: .NET MAUI

9.2 Scalability Plan

Stage	Architecture
MVP	Monolith
Growth	Cache + replicas

Scale	Microservices

---

10. OPERATIONS PLAN

10.1 Team (Initial)

Role	Responsibility
Founder	Product + strategy
Developer	Backend + frontend
Marketing	Outreach + content

10.2 Daily Execution

- Agent outreach
- Listing upload
- Content creation
- Performance tracking

---

11. RISKS & MITIGATION

11.1 Business Risks

Risk	Mitigation
No listings	Manual onboarding
Low traffic	SEO + ads
Low conversion	Improve UX

11.2 Technical Risks

Risk	Mitigation
Slow search	Elastic
Scaling issues	Redis + CDN

---

12. ROADMAP

Phase 1 (0–1 Month)

- MVP launch
- 500 listings

Phase 2 (1–3 Months)

- AI recommendation
- Monetization

Phase 3 (3–6 Months)

- Scale infrastructure
- Expand region

---

13. SUCCESS METRICS

North Star Metric

👉 **Number of leads generated per day**

Supporting Metrics

- Listings count
- Active users
- Conversion rate
- Agent retention